BOOTCAMP III · FASE 3 · ENTREGA FINAL

System Design Document

TaskAnalyzer — Módulo de Análise de Tarefas e Produtividade

Especificação executável sob o paradigma **Spec-Driven Development (SDD)**: contrato de negócio, cenários de aceite, governança de IA e arquitetura de entrega contínua (CI/CD + Docker).

Campo	Valor
Autor	Marcos Vogado
Repositório	github.com/MarcosVogado/sdd-task-analyzer
Versão da especificação	3.0 (Fase 3)
Data	Outubro de 2026
Artefatos relacionados	`specs/task_analyzer_spec.md` · `specs/requirements.yaml` · `CONTEXT_RULES.md`

Princípio orientador: a especificação é o contrato e a fonte da verdade; a IA é a executora; o engenheiro é o arquiteto e o homologador.

1. Visão Geral e Propósito

O TaskAnalyzer é um módulo **puramente analítico** que recebe uma lista de tarefas e retorna métricas de produtividade: contagens, tempo médio de conclusão em horas, taxa de atraso e indicadores por prioridade. O módulo **não persiste dados** e **não possui interface gráfica** — é uma unidade de lógica de negócio testável, reutilizável e auditável.

2. Contrato Executável de Interface

2.1 Função pública

```
def analyze_tasks(tarefas: list[Tarefa]) -> dict[str, Any]: ...
class TaskValidationError(Exception): ...
```

2.2 Entrada — Tarefa

Campo	Tipo	Obrig.	Restrições
`id_tarefa`	int	Sim	Maior que zero.
`prioridade`	str (enum)	Sim	baixa · media · alta
`status`	str (enum)	Sim	concluida · pendente · cancelada
`data_criacao`	datetime (UTC)	Sim	—
`prazo`	datetime (UTC)	Sim	—
`titulo`	str | None	Não	—
`data_inicio`	datetime | None	Cond.	Obrigatória quando status = concluida.
`data_conclusao`	datetime | None	Cond.	Obrigatória quando status = concluida.

2.3 Saída

Campo	Tipo	Descrição
`total_tarefas`	int	Total de tarefas recebidas.
`total_concluidas`	int	Tarefas com status concluida.
`total_pendentes`	int	Tarefas com status pendente.
`tempo_medio_conclusao_horas`	float	Média, em horas, das concluídas.
`taxa_atraso_percentual`	float	Percentual de concluídas entregues após o prazo.
`indicadores_por_prioridade`	dict	Tempo médio e taxa de atraso por prioridade.

Todas as métricas numéricas são arredondadas em 2 casas decimais.

3. Regras de Negócio e Validação

ID	Regra
RN01	Apenas tarefas concluídas entram em tempo médio e taxa de atraso.
RN02	Tarefas canceladas são ignoradas nas métricas (contam apenas em total_tarefas).
RN03	Tarefas pendentes entram somente nas contagens.
RN04	Entrada vazia ou sem concluídas retorna 0.0 nas médias e taxas, sem exceção (prevenção de divisão por zero).

ID	Regra
RN05	Datas inconsistentes (início antes da criação, conclusão antes da criação ou do início) disparam TaskValidationError.
RN06	Prioridade ou status fora do enum disparam TaskValidationError.
RN07	Tarefa concluída sem data_inicio ou data_conclusao dispara TaskValidationError.
—	Tempo de conclusão = data_conclusao − data_inicio, em horas; atraso quando data_conclusao > prazo; datas em UTC.

4. Cenários de Aceite e Rastreabilidade

Cada requisito é ligado a pelo menos um teste automatizado do Test Harness. A versão legível por máquina dessa matriz está em `specs/requirements.yaml`; a cobertura é de **100%** (nenhum requisito sem teste e nenhum teste sem requisito).

Cenário (Dado — Quando — Então)	Requisitos	Testes (tests/test_harness.py)
CA01 — Sucesso. Dado tarefas válidas, quando analyze_tasks executa, então retorna contagens e métricas corretas, geral e por prioridade.	RF01–RF04, RN01	`test_cenario1_metricas_gerais` `test_cenario1_indicadores_por_prioridade`

CA02 — Exceção. Dado datas inconsistentes ou enum inválido, então dispara TaskValidationError com mensagem clara.	RN05–RN07	`test_cenario2_datas_inconsistentes`
		`test_cenario2_status_invalido`
		`test_cenario2_prioridade_invalida`
		`test_cenario2_concluida_sem_datas`

CA03 — Canceladas ignoradas.	RN02	`test_canceladas_sao_ignoradas`
CA04 — Atraso contabilizado.	RF03	`test_atraso_contabilizado_corretamente`
CA05 — Bordas. Lista vazia e apenas pendentes retornam 0.0.	RN03, RN04	`test_borda_lista_vazia` `test_borda_apenas_pendentes`

5. Arquitetura do Repositório

```
sdd-task-analyzer/
├── .github/workflows/ci.yml      # Motor CI/CD (GitHub Actions)
├── Dockerfile                    # Empacotamento (python:3.11-slim)
├── docker-compose.yml            # Orquestração (tests, demo)
├── CONTEXT_RULES.md              # Base de governança da IA (Fase 1)
├── specs/
│   ├── system_design_document.pdf  # Este documento
│   ├── requirements.yaml           # Requisitos rastreáveis
│   └── task_analyzer_spec.md       # Contrato em texto
├── tests/test_harness.py         # Suíte de testes (Fase 2)
├── src/task_analyzer.py          # Código gerado via IA e homologado
├── requirements.txt              # Dependências autorizadas (fixadas)
└── README.md
```

6. Governança de IA (CONTEXT_RULES)

As regras abaixo são fornecidas ao assistente de IA em toda solicitação e funcionam como um *firewall* entre a geração de código e o repositório.

Diretrizes obrigatórias

- Python 3.11+, type hints em 100% das funções, PEP 8 e SRP.

- Docstrings no padrão Google; funções pequenas e coesas.
- Exceção única e específica: TaskValidationError, com mensagens claras.
- Logs estruturados com o módulo padrão logging.

Proibições explícitas

- Bibliotecas externas não autorizadas; persistência de dados.
- Alterar cenários de teste, estrutura de pastas ou a assinatura de analyze_tasks.
- Assumir comportamento não descrito no contrato.
- **Fase 3:** desabilitar ou pular testes para obter build verde; alterar pipeline/Docker fora de PR; dependências sem versão fixada; segredos no repositório.

7. Infraestrutura de Entrega

7.1 Pipeline de Integração Contínua (GitHub Actions)

Etapa	Definição
Gatilhos	push e pull_request nas branches main e develop.
Ambiente	ubuntu-latest com Python 3.11 (actions/setup-python).
Preparação	pip install -r requirements.txt (pytest com versão fixada).
Execução	pytest tests/test_harness.py -v
Validação do container	Job docker: build da imagem e execução da suíte dentro dela (depende do job de testes).
Política	Build vermelho bloqueia o merge; nenhum teste pode ser pulado.

7.2 Containerização (Docker)

Artefato	Responsabilidade
Dockerfile	Empacotamento: base python:3.11-slim, instala dependências, copia apenas src/, tests/ e conftest.py, executa como usuário não-root; CMD = pytest tests/test_harness.py -v.
docker-compose.yml	Orquestração: serviço tests (suíte completa) e demo (análise de exemplo), acionados por docker compose run --rm <serviço>.
.dockerignore	Mantém a imagem enxuta: exclui .git, ambientes virtuais, caches e documentação.

8. Fluxo de Entrega e Homologação (Critical-in-the-Loop)

Etapa	Responsável	Barreira
1. Geração	Assistente de IA	Recebe spec + CONTEXT_RULES; gera código em branch feature/*.
2. Barreira SDD	Contrato + testes	Test Harness local e CI verificam aderência ao contrato.
3. Validação humana	Engenheiro (homologador)	Revisão crítica do PR: contrato, regras, qualidade e casos de borda.
4. Integração segura	Pipeline	Merge em develop com CI verde; release para main; tag de versão.

```
feature/*  ──PR + CI──▶  develop  ──PR de release + CI──▶  main  ──▶  tag v1.0.0
```

9. Histórico de Versões da Especificação

A especificação evoluiu de forma controlada; cada mudança foi registrada, aplicada primeiro ao contrato e só depois ao código.

Versão	Fase	Mudanças
1.0	Fase 1	Contrato inicial: tempo em minutos; exceções EntradaInvalidaError e SemTarefasConcluidasError; ausência de concluídas tratada como exceção.
2.0	Fase 2	Alinhamento aos requisitos da Fase 2: função analyze_tasks, exceção única TaskValidationError, tempo em horas, contagens total_concluidas/total_pendentes e retorno 0.0 para entrada vazia.
3.0	Fase 3	Requisitos rastreáveis (requirements.yaml), CI/CD, Docker, dependências fixadas e regras de governança de pipeline.